תרגיל בית 5 (ואחרון!)

numpy, pandas, image processing

<u>הנחיות כלליות:</u>

- קראו היטב את השאלות והקפידו שהתכניות שלכם פועלות בהתאם לנדרש.
- את התרגיל יש לפתור לבד!
- אין לשנות את שמות הפונקציות והמשתנים שכבר מופיעים בקובץ השלד של התרגיל.
- אין למחוק את ההערות שמופיעות בשלד.
- יש לבצע בדיקה עצמית אך ורק בסוף קובץ השלד בתוך התנאי הבא:

  if __name__ == "__main__":

  בפרט אין להשאיר הדפסות/קריאות לפונקציות בין הפונקציות.
- אין להשאיר בקוד נתיבים לקבצים מקומיים.
- הקפידו על כללי ההגשה המפורסמים באתר. בפרט, יש להגיש את כל השאלות יחד בקובץ ex5_012345678.py המצורף לתרגיל, לאחר החלפת הספרות 012345678 במספר ת.ז שלכם, כל 9 הספרות כולל ספרת הביקורת.
- <u>אופן ביצוע התרגיל:</u> בתרגיל זה עליכם לממש את הפונקציות הנתונות. ניתן להוסיף פונקציות עזר.
- מועד אחרון להגשה: כמפורסם באתר.
- היות ובדיקת התרגילים עשויה להיות אוטומטית, יש להקפיד על פלטים מדויקים על פי הדוגמאות (עד לרמת הרווח).
- <u>בדיקה עצמית:</u> כדי לוודא את נכונותן ואת עמידותן של התוכניות לקלטים שגויים, בכל שאלה, הריצו את תוכניתכם עם מגוון קלטים שונים, אלה שהופיעו כדוגמאות בתרגיל וקלטים נוספים עליהם חשבתם (וודאו כי הפלט נכון וכי התוכנית אינה קורסת).
- בכל השאלות ניתן להניח את תקינות הקלט על פי המפורט בשאלה.

שאלה 1 numpy –

בשאלה נעסוק בטבלת מידע המתארת את תוצאות הניסוי של תרופת פלא חדשה לירידה במשקל. במשך מספר חודשים נאספו על-ידי ד"ר רוסום נתונים של מדידות משקל חודשיות עבור מספר נסיינים, כך שבעמודה הראשונה מדידות משקל של כל הנסיינים בחודש הראשון לניסוי, בעמודה השנייה המדידות בחודש השני, וכך הלאה.

במהלך השאלה ניתן להניח שמקבלים את טבלת המדידות כמטריצת numpy ללא שמות של חודשים או משתתפים. אלה יסופקו לפי הצורך בנפרד. כמו כן, במהלך השאלה לא ניתן להשתמש בלולאות כלל. במקום זאת, נשתמש באפשרויות שמספקת לנו ספריית numpy.

<u>טבלה לדוגמא (רק התאים המודגשים הם חלק ממטריצת המדידות) :</u>

participant_names / month	November	December	January	February	March	April
Tali	**92.3**	**91.5**	**89.4**	**89.2**	**88.6**	**85.9**
Avi	**104.6**	**102.1**	**100.8**	**98.5**	**96.3**	**94.4**
Naomi	**73.2**	**72.6**	**72.0**	**71.4**	**71.2**	**70.9**
Shlomi	**78.3**	**78.0**	**77.2**	**75.8**	**74.7**	**74.4**

1. יש לממש את הפונקציה get_highest_weight_loss_participant(training_data, participant_names), אשר מקבלת מטריצת numpy של המדידות (training_data) וכן רשימת שמות המשתתפים בניסוי (participant_names). הפונקציה תמצא את המשתתף שירידתו במשקל מתחילת התכנית ועד הסוף הייתה הכי גדולה. יש לממש את הפונקציה ללא שימוש בלולאות, בשתי שורות לכל היותר.

הערות:

- סדר השורות המטריצה תואם את סדר המשתתפים ברשימה. כלומר, השורה ה-i במטריצה שייכת למשתתף ה-i ברשימת המשתתפים בניסוי.
- ניתן להניח שתמיד קיים משתתף יחיד שערך ירידתו במשקל היא המקסימלית.
- שימו לב, ניתן (אבל לא חובה) להשתמש בפונקציה numpy.argmax, אשר מחזירה את האינדקס בו נמצא הערך המקסימלי.

<u>דוגמת הרצה על הטבלה למעלה:</u>

```
>>> get_highest_weight_loss_participant(training_data, participant_names)
'Avi'
```

2. נגדיר את "הפרש חודשי" כהפרש המשקל של משתתף מסוים בין **חודש לחודש שלפניו**. יש לממש את הפונקציה get_diff_data(training_data), אשר מקבלת את מטריצת המדידות החודשיות training_data ומחזירה את מטריצת ההפרשים, כך **שבעמודה ה-i יופיע המשקל בחודש ה-i+1 פחות המשקל בחודש ה-i**, ומספר העמודות סה"כ יהיה קטן ב-1 ממספר העמודות במטריצת הקלט. יש לממש את הפונקציה ללא שימוש בלולאות, בשורה אחת בלבד.

<u>דוגמת הרצה על הטבלה למעלה:</u>

```
>>> get_diff_data(training_data)
array([[-0.8, -2.1, -0.2, -0.6, -2.7],
       [-2.5, -1.3, -2.3, -2.2, -1.9],
       [-0.6, -0.6, -0.6, -0.2, -0.3],
       [-0.3, -0.8, -1.4, -1.1, -0.3]])
```

3. למעשה, תרופת הפלא ד"ר רוסום לא אמורה לעודד ירידה במשקל, אלא למתן ירידה/עלייה במשקל כך שתהיה קבועה. כלומר, המטרה היא שבכל חודש נסיינים ירדו או יעלו אותה כמות במשקל.

כך למשל, עבור הנסיין שלומי, אם התחיל את הניסוי במשקל 78.3 קילוגרם ולאחר שישה חודשים היה במשקל 74.4 קילוגרם, מטרת התרופה שיירד בכל חודש 780 גרם בדיוק.

יש לממש את הפונקציה get_distance_from_linear_change(training_data), אשר מקבלת את המדידות החודשיות training_data ומחזירה את ההפרש בין המשקלים שנמדדו למשקלים המצופים בכל נקודת אמצע מתחילת הניסוי ועד סופו.

<u>רמז:</u> הפונקציה numpy.linspace שראינו בהרצאה יכולה לעבוד גם על numpy.arrays.

<u>דוגמת הרצה על הטבלה למעלה:</u>

```
>>> get_distance_from_linear_change(training_data)
array([[ 0.48, -0.34,  0.74,  1.42],
       [-0.46,  0.28,  0.02, -0.14],
       [-0.14, -0.28, -0.42, -0.16],
       [ 0.48,  0.46, -0.16, -0.48]])
```

שאלה 2 – עיבוד תמונה

בתרגיל זה נממש קוד להסתרה של הודעות טקסט קצרות בתוך תמונות. שיטה זו להעברה חשאית של הודעות בתוך תמונות תמימות ידועה בשם - סטגנוגרפיה (steganography).

https://en.wikipedia.org/wiki/Steganography

כמו שראינו בכיתה, ניתן להשתמש במערכים דו מימדיים של Numpy לייצוג תמונות בגווני אפור. כל איבר במערך הדו-מימדי הוא מספר שלם המייצג את גוון האפור של פיקסל אחד בתמונה. בתרגיל זה הטיפוס של כל איבר במערך יהיה np.uint8 (מספר שלם שמקודד בבייט בודד), המסוגל לייצג ערכים בטווח 0-255 ועל כן ניתן לייצג איתו 256 גווני אפור, אך גם אפשר לתת לו משמעות של תו במחרוזת כלשהי. אנחנו ננצל את התכונה הזאת כדי להסתיר הודעות בתוך תמונות.

עבור תמונה נתונה ומחרוזת קצרה כלשהי, נכתוב קוד שיחזיר תמונה חדשה בה מקודדת ההודעה באופן שישפיע כמה שפחות על מראה התמונה. כדי שיהיה ניתן לחלץ את ההודעה מהתמונה, נשמור במערך המייצג את התמונה גם נתונים נוספים: המיקום בתמונה בה שולבה ההודעה, ואורך המחרוזת.

כיצד נשלב הודעה (רצף של מספרים המייצגים תווים) בתוך תמונה בגווני אפור (המיוצגת על ידי מערך דו מימדי של מספרים) בצורה הסודית ביותר? נתחיל בתיאור כללי של שלבי האלגוריתם:

1. האלגוריתם אותו נממש יתחיל בהמרת טקסט ההודעה לרצף של מספרים תוך שימוש בפונקציית **ord()**, המחזירה לכל תו בודד את ערך ה-ASCII שלו.
לדוגמא:

```
<<< print(ord('H'), ord('e'), ord('l'), ord('l'), ord('o'))
```

72 101 108 108 111

בקובץ התרגיל כבר נתונות לכם שתי פונקציות עזר שמבצעות המרה ממחרוזת טקסט למערך Numpy של מספרים מסוג 8np.uint המכיל את ערכי ה-ASCII של כל תו ולהיפך. שמות הפונקציות: **np_array_to_ascii**-ו**ascii_to_np_array**.
דוגמת שימוש:

```
>>> x = ascii_to_np_array("hello")
>>> x
array([104, 101, 108, 108, 111], dtype=uint8)
>>> np_array_to_ascii(x)
'hello'
```

2. האלגוריתם ימשיך במציאת מיקום (זוג אינדקסים - שורה ועמודה) במערך התמונה שבו מתחיל רצף של מספרים שהינו בעל רמת הדימיון הגבוהה ביותר לרצף המספרים המקודדים את הודעת הטקסט. החלפת רצף מספרים זה במערך התמונה עם רצף המספרים המקודדים את ההודעה ישפיע בצורה מינימלית על מראה התמונה. מיקום תחילת ההודעה אינו יכול להכיל אינדקס שערכו גדול מ-255 (ראו הערות). כמו כן, על ההודעה להיות מוכלת בשורה בודדת של התמונה. משמע, אם אורך ההודעה הוא k, אזי לא ניתן למקם את נקודת ההתחלה שלה פחות מ-k-1 פיקסלים מהקצה הימני של התמונה. ניתן להניח כי הקלט שתקבלו יעמוד בתנאים אלו.

3. ליצירת תמונה בה משולבת ההודעה, נבצע את הצעדים הבאים:
   3.1. נעתיק את מטריצת הקלט שמייצגת את התמונה למטריצה חדשה בה ניצור את תמונת הפלט.
   3.2. נמצא את המיקום הכי טוב בתמונה לשתילת ההודעה.
   3.3. נעתיק את התוצאה אל המיקום שמצאנו בסעיף הקודם.
   3.4. נשמור מידע נוסף בפיקסלים בפינה השמאלית העליונה במערך תמונת הפלט שיאפשר חילוף של ההודעה בעתיד:
   פיקסל [0,0] – אינדקס השורה אליה הועתקה ההודעה
   פיקסל [0,1] – אינדקס העמודה אליה הועתקה ההודעה
   פיקסל [0,2] – אורך ההודעה

הערות

1. התמונה אינה בהכרח ריבועית.
2. ניתן להניח שאורך ההודעה קטן ממספר העמודות בתמונה.
3. ניתן להניח שההודעה אינה ריקה.
4. היות וכל איבר בתמונה הוא מסוג np.uint8 שיכול לייצג מספרים בטווח 0-255 בלבד, והיות ומיקום תחילת ההודעה נשתל בעצמו כפיקסל של התמונה, ניתן לשתול את ההודעה רק באינדקסים הקטנים מ-256.

5. ניתן להניח כי מיקום ההודעה לא יהיה כזה שבו נתוני ההודעה ידרסו את אחד משלושת הפיקסלים הראשונים בשורה הראשונה, מה שהיה עלול לפגוע במידע הנוסף.

1. ממשו פונקציה **arr_dist(a1, a2)** המקבלת שני מערכי numpy, ומחשבת את סכום המרחקים בין שני המערכים כסכום של ערכים מוחלטים של הפרשים בין האיברים מתאימים. יש להניח שגודל המערכים זהה.

הערות:

- יש לממש את הפונקציה ללא שימוש בלולאות, בשורה אחת בלבד.

דוגמה: המרחק בין המחרוזת "Hello" והמחרוזת "Halmo" הוא 5 .ערכי הASCII של התוים מופיעים בסוגריים.

H (72)	**e** (101)	**l** (108)	**l** (108)	**o** (111)
H (72)	**a** (97)	**l** (108)	**m** (109)	**o** (111)
72-72	|101-97|	108-108	|108-109|	111-111
0	4	0	1	0

```
>>> a1 = ascii_to_np_array('Hello')

>>> print(a1)

[ 72 101 108 108 111 ]

>>> a2 = ascii_to_np_array('Halmo')

>>> print(arr_dist(a1, a2))

5
```

2. ממשו פונקציה **find_best_place(im, np_msg)** המקבלת את מטריצת התמונה (im) ומערך ההודעה (np_msg) ומחזירה כ-tuple זוג אינדקסים – שורה ועמודה – של המיקום בתמונה שבו מתחיל רצף ערכים (באותו אורך כמו של אורך ההודעה) שהכי דומה למערך ההודעה. כלומר, ככל שהמרחק בין רצף הערכים לבין ההודעה (כפי שמתקבל מהפונקציה בשאלה 1) יותר קטן, הוא יותר דומה לה.

הערות:

- הtuple בערך ההחזר יכיל 2 איברים בדיוק: האיבר הראשון הוא אינדקס השורה והאיבר השני הוא אינדקס העמודה.
- ניתן להשתמש בלולאות.

זכרו כי יש לוודא שההודעה מוכלת כולה בשורה אחת של התמונה (ראו תאור האלגוריתם למעלה).

דוגמה:

```
>>> im1 = imageio.imread('parrot.png')

>>> np_msg = ascii_to_np_array('Hello')

>>> idx = find_best_place(im1, np_msg)
```

```
>>> print(idx, type(idx))
```

(110, 121) <class 'tuple'>

3. ממשו פונקציה **create_image_with_msg(im, im_idx, np_msg)** המקבלת את מטריצת התמונה (im), tuple המכיל זוג האינדקסים של מיקום בתמונה (im_idx) ואת מערך ההודעה (np_msg). הפונקציה תחזיר מערך דו מימדי המייצג את תמונת הפלט בו שתולה ההודעה.

הפונקציה תבצע את השלבים הבאים:

1 . יצירת עותק של מערך התמונה (ניתן להשתמש במתודה copy הפועלת על מערך של numpy) על מנת שמערך הקלט המקורי לא ישתנה.

2 .העתקת מערך ההודעה למיקום המוגדר ע"י האינדקסים idx_img (שורה ועמודה, כtuple).

3 .שמירת ערכו של אינדקס השורה בה מתחילה ההודעה במיקום [0,0] בתמונה החדשה.

4 .שמירת ערכו של אינדקס העמודה בה מתחילה ההודעה במיקום [**0,1**] בתמונה החדשה.

5 .שמירת אורך ההודעה במיקום [**0,2**] בתמונה החדשה.

6 .החזרת המערך הדו מימדי שמייצג את התמונה חדשה.

הערות:

- אין להשתמש בלולאות

דוגמה: בהמשך לדוגמה של שאלה 2 ,הפונקציה תשתול את ההודעה באופן הבא בתמונת הקלט:

הפיקסלים הצהובים (בתוך התמונה) – תוכן ההודעה, פיקסלים הירוקים בפינה השמאלית העליונה – מידע עזר, ושאר הפיקסלים ישארו כמו בתמונה המקורית.

4. ממשו פונקציה **put_message(im, msg)** המקבלת תמונה (im) ומחרוזת ההודעה (msg) ומחזירה תמונה עם מחרוזת נסתרת בפנים. הפונקציה מבצעת את השלבים הבאים:

1. הופכת את המחרוזת למערך.
2. מוצאת את המקום הכי מתאים בתמונה (באמצעות הפונקציה של שאלה 2).

3. בונה תמונה חדשה (באמצעות הפונקציה של שאלה 3).
4. מחזירה תמונה חדשה (מסוג מטריצת numpy).

הערות:

- אין להשתמש בלולאות

5. ממשו פונקציה **get_message(im)** שמחזירה את טקסט ההודעה השתולה בתמונת הקלט. יש להניח שהודעה קיימת.

הפונקציה מבצעת השלבים הבאים:

1. שולפת פיקסלים של הודעה כמערך באמצעות שלושת הפיקסלים השתולים כעזר בתמונה.
2. מחזירה את מחרוזת הטקסט של ההודעה השתולה (יש לקרוא לפונקצית **np_array_to_ascii** הממומשת).

הערות:

- אין להשתמש בלולאות

דוגמה:

```
>>> im1 = imageio.imread('parrot.png')

>>> enc_image = put_message(im1, "hello pyprog")

>>> print(get_message(enc_image))

hello pyprog
```

שאלה 3 – pandas

שימו לב: בכל סעיפים השאלה זו אין להשתמש בלולאות

אתר השירות המטאורולוגי מספק מידע מסוגים שונים לציבור. אחרי החודש הגשום האחרון, קיבלנו נוסטלגיה לתקופה שבה חורף היה יבש וחם כמו שהוא צריך להיות - דצמבר 2023.

בשאלה זו נבדוק האם דצמבר באמת היה חם כמו שאנחנו זוכרים בעזרת pandas, על-ידי בדיקת הטמפרטורות שנמדדו בחודש דצמבר בשנת 2023. לדוגמא:

day	netanya	beer_sheva	tel_aviv	safad
3	26.5	22.6	77	24.4
4	26.7	26.2	79.16	25.9
5	23.6	23.3	73.76	24.8
6	22	19.4	71.6	NaN
7	22.3	18.9	68.72	19.9
8	24.7	20.7	72.5	22.1
9	21.8	17.7	65.84	18.1
10	22.8	18.9	67.46	20.2
11	23.3	20.9	73.22	22.6
12	22.6	18.4	70.34	21.1
13	21.5	15.2	67.46	19.6
14	22.6	17.9	69.08	20.5
15	23.4	18.3	67.64	19.9
16	23.4	22.8	69.08	20.7

1. ממשו את הפונקציה load_weather_csv(file_name), הפונקציה מקבלת את שמו של קובץ המידע כפי שהוגדר לעיל (מחרוזת), ותחזיר DataFrame של החבילה **pandas** כך שכל עמודה תכיל את המדידות בעיר מסוימת במהלך החודש (פרט לעמודה הראשונה שמכילה את היום בחודש), וכל שורה תכיל את כל המדידות שנעשו ביום מסוים. התוצאה צריכה להיות בדומה לטבלה למעלה. ניתן להניח כי ניתן הקובץ קיים, שערכיו תקינים ומכילים לפחות שורת מדידות אחת (ושורת כותרות לעמודות).
הערות:
   - יש לממש את הפתרון בשורה אחת בלבד

2. בשאלה זו נעסוק בבעיה חשובה בעיבוד מידע: ניקוי ערכים לא תקינים ותיקונם (https://en.wikipedia.org/wiki/Data_cleansing).

א. ממשו את הפונקציה impute_to_mean(weather_data), המקבלת את הטבלת שהחוזרה בסעיף הקודם (weather_data). הפונקציה מחזירה שכפול של הטבלה בו ערכים ריקים בטבלה המקורית מוחלפים בממוצע הטמפרטורה בעיר לאותו חודש.
הערות:
- יש לממש את הפתרון בשתי שורות לכל היותר

לדוגמא, הפעלת הפונקציה על הטבלה למעלה תחליף את הערך NaN ב-21.52, הממוצע בעיר צפת.

ב. חלק מחיישני המדידה של השירות המטאורולוגי נפגמו, ומדדו את הטמפרטורות רק ביחידות פרנהייט במשך כל החודש. ניתן לזהות מקומות בהם קיימת התקלה ע"י מדידות גבוהות מ-40 מעלות, החורף פה חם, אבל לא כזה חם.
נממש את הפונקציה fix_to_celsius אשר תמצא את הערים בהן אירעה התקלה, ותחזיר טבלה בה כל הערכים במעלות צלזיוס. <u>רמז:</u> הנוסחה להמרה בין צלזיוס לפרנהייט היא:

$$C = \frac{(F - 32) \times 5}{9}$$

לדוגמא, הפעלת הפונקציה על הטבלה למעלה אמורה לתקן רק את הערכים שנמדדו בתל-אביב ולתקנם כך שב-3.12 (היום הראשון למדידה) הטמפרטורה תהיה 25 מעלות, ב-4.12 תהיה 26.2 וכן הלאה. ניתן לראות את התיקון המלא בדוגמת הרצה לשאלה 3א למטה.
הערות:

- יש לממש את הפתרון בשתי שורות לכל היותר

<u>שימו לב:</u>

- בסעיפים הבאים יש לתקן את הטבלה המקורית בתחילת השאלה.
  ניתן (אך לא חייבים) להיעזר בפונקציה הנתונה **clean_data** אשר מפעילה את שני סעיפי שאלה 2 על טבלת DataFrame המקורית המועברת כארגומנט, ומחזירה טבלת DataFrame מתוקנת.
- יש לממש כל פתרון בארבע שורות לכל היותר

3. בדיקה יומית של הטמפרטורה היא מדידה קצת דקה מדי עבורנו. נרצה למדוד את ממוצע המעלות השבועי במקום.
   א. ממשו את הפונקציה add_week_index, אשר מחזירה טבלה עם עמודה נוספת בה מופיע ערך השבוע מאז תחילת המדידה, כאשר מתחילים לספור מ-1. דוגמת הרצה על הטבלה למעלה:

```
>>> add_week_index(example)
    day  netanya  beer_sheva  tel_aviv  safad  week
0     3     26.5        22.6      25.0   24.4     1
1     4     26.7        26.2      26.2   25.9     1
2     5     23.6        23.3      23.2   24.8     1
3     6     22.0        19.4      22.0  21.52     1
4     7     22.3        18.9      20.4   19.9     1
5     8     24.7        20.7      22.5   22.1     1
6     9     21.8        17.7      18.8   18.1     1
7    10     22.8        18.9      19.7   20.2     2
8    11     23.3        20.9      22.9   22.6     2
9    12     22.6        18.4      21.3   21.1     2
10   13     21.5        15.2      19.7   19.6     2
11   14     22.6        17.9      20.6   20.5     2
12   15     23.4        18.3      19.8   19.9     2
13   16     23.4        22.8      20.6   20.7     2
```

ב. ממשו את הפונקציה get_weekly_mean, אשר מקבלת טבלת מדידות ומחזירה טבלה חדשה עם ממוצע הטמפרטורות השבועי בכל עיר.
דוגמת הרצה על הטבלה למעלה:

```
>>> get_weekly_mean(example)
        netanya  beer_sheva   tel_aviv      safad
week
1     23.942857   21.257143  22.585714  22.389011
2     22.800000   18.914286  20.657143  20.657143
```

4. לקרבה למקורות מים גדולים אמורה להיות השפעה ממתנת על משרעת הטמפרטורות. נרצה למדוד השפעה זו.
א. ממשו את הפונקציה get_temperature_range, אשר מקבלת טבלת מדידות חודשית ומחזירה את משרעת הטמפרטורות שנמדדו באותו חודש עבור כל עיר (כלומר, ההפרש בין טמפ' מקסימלית לטמפ' מינימלית).
דוגמת הרצה על הטבלה למעלה:

```
>>> get_temperature_range(example)
netanya        5.2
beer_sheva    11.0
tel_aviv       7.4
safad          7.8
dtype: float64
```

ב. ממשו את הפונקציה find_coastal_effect(weather_data, coastal_cities), אשר מקבלת טבלת מדידות חודשית (weather_data) ורשימת מחרוזות של שמות ערי חוף (coastal_cities), ומחזירה את ההפרש בין

המשרעת הממוצעת בערי חוף פחות המשרעת הממוצעת בערים רחוקות מהחוף. אם תוצאת ההפרש היא שלילית, זו אינדיקציה שהקרבה לים באמת ממתנת את הטמפרטורות.
דוגמת הרצה על הטבלה למעלה:

```
>>> find_coastal_effect(weather_data, coastal_cities)
-3.1000000000000005
```

5. יש לנו רק מדידות של טמפרטורות, אבל מידע מוקדם ניתן להניח שבימים בהם הטמפרטורה הייתה נמוכה מ-20 מעלות היה גם גשם. ממשו את הפונקציה add_rainy_days, אשר מקבלת טבלת מדידות חודשית ומחזירה טבלה עם שורה נוספת בתחתית, עם מספר ימי הגשם שהיו בחודש בכל עיר. שימו לב, לעמודה day מספר ימי הגשם צריך להיות 0.
דוגמת הרצה על הטבלה למעלה:

```
>>> add_rainy_days(example)
            day  netanya  beer_sheva  tel_aviv      safad
0             3     26.5        22.6      25.0  24.400000
1             4     26.7        26.2      26.2  25.900000
2             5     23.6        23.3      23.2  24.800000
3             6     22.0        19.4      22.0  21.523077
4             7     22.3        18.9      20.4  19.900000
5             8     24.7        20.7      22.5  22.100000
6             9     21.8        17.7      18.8  18.100000
7            10     22.8        18.9      19.7  20.200000
8            11     23.3        20.9      22.9  22.600000
9            12     22.6        18.4      21.3  21.100000
10           13     21.5        15.2      19.7  19.600000
11           14     22.6        17.9      20.6  20.500000
12           15     23.4        18.3      19.8  19.900000
13           16     23.4        22.8      20.6  20.700000
Rainy Days    0      0.0         8.0       4.0   4.000000
```

בהצלחה!